



A multi-agent system for FJSP with setup and transportation times

Manojkumar Pal ^{a,*}, Murari Lal Mittal ^a, Gunjan Soni ^a, Satyendra S. Chouhan ^b, Manish Kumar ^a

^a Department of Mechanical Engineering, MNIT Jaipur, 302017, India

^b Department of Computer Science Engineering, MNIT Jaipur, 302017, India

ARTICLE INFO

Keywords:

Flexible job shop scheduling
Multi-agent system
Setup time
Transportation time
Grey wolf algorithm
Decentralized system

ABSTRACT

Flexible job shops are quite common in manufacturing, where the machines can perform a variety of operations on a job. The setup and transportation times are quite significant in flexible job shops, which recently have attracted attention. This paper deals with the problem of scheduling of flexible job shops, termed FJSP, with setup and transportation times. Several metaheuristics approaches have been proposed for the problem assuming centralized decision-making. The centralized approaches, however, may not be suitable for large complex problems due to higher computational effort and slower convergence. The decentralized approaches are more suitable for such problems. To the best of authors' knowledge, no work has been reported towards developing a decentralized approach for FJSP with setup and transportation times. To fill this gap a multi-agent system, a popular decentralized approach, is developed for FJSP with setup and transportation time. The performance of the proposed approach is compared with the three state-of-the-art (centralized) approaches by solving 20 problem instances. The proposed approach has been found to produce schedules with makespan lower (maximum of 35% and an average of 10.27%) than the comparison approaches. The algorithm has also been found to converge faster than the comparison algorithms.

1. Introduction

Job shop scheduling problem (JSP) is a widely researched problem in production scheduling. It aims at determining an optimum schedule of jobs with predefined routes. The machines here are assumed to perform specific operations, which makes the system inflexible. Nevertheless, the manufacturing system must be flexible to fulfil varying customer demands and market requirements (Fattahi et al., 2009). This flexibility can be achieved with multipurpose machines (e.g. machining centres), which can perform a variety of operations on a job. The job shops with flexible machines are generally known as Flexible Job Shops (FJSs). The scheduling of FJS, however, is more challenging than JSP due to the additional dimension of machine selection. The problem of scheduling of FJS is termed as Flexible Job-shop Scheduling Problem (FJSP) in literature.

A variety of objective functions and constraints have been considered in FJSP. The most common objective for solving FJSP is minimizing the makespan. The main constraints in FJSP are job precedence and avoidance of multiple operations at a machine at the same time. Recently, to represent the real issues of FJSP, factors such as machine setup time and transportation time have been considered in FJSP. Machine setup time is vital in a high variety production where a large number of setups are required. Transportation times count for the

transfer of jobs between machines. If not considered, the setup time and transportation time may render the schedules infeasible or sub-optimal (Nouri et al., 2016; Zhang et al., 2020). Operation time, setup and transportation times are considered as independent parameters for the problem (Li, Deng, et al., 2020; Zhang et al., 2020).

JSP is an NP-hard problem, and so is the FJSP. Hence the exact methods are not suitable for such large-sized complex problems. Thus, a variety of metaheuristic approaches such as Genetic algorithm (GA), Jaya algorithm, Evolutionary algorithm, Tabu search (TS), Simulated annealing (SA), Imperialist competitive algorithm (ICA) have been proposed. Most of these existing approaches assume a centralized decision-making environment where a single decision-maker, who has access to all the system information, takes decisions for scheduling. These centralized approaches, however, suffer from higher computational load and lower convergence with an increase in size and complexity of the scheduling problems (Zhang, Ding, et al., 2019). On the other hand, the decentralized approaches, where a problem is decomposed into several smaller sub-problems, are very effective in handling sizeable complex scheduling problems and can lead to optimal scheduling (Barbati et al., 2012; Xiong & Fu, 2018). Authors in Zhang, Ding, et al. (2019) advocate the use of decentralized decision-making

* Corresponding author.

E-mail addresses: 2017rme9086@mnit.ac.in (M. Pal), mlmittal.mech@mnit.ac.in (M.L. Mittal), gsoni.mech@mnit.ac.in (G. Soni), sschouhan.cse@mnit.ac.in (S.S. Chouhan), m81manishkumar@gmail.com (M. Kumar).

<https://doi.org/10.1016/j.eswa.2022.119474>

Received 26 September 2022; Received in revised form 15 November 2022; Accepted 22 December 2022

Available online 29 December 2022

0957-4174/© 2022 Published by Elsevier Ltd.

for FJSP. Decentralized decision-making is critical for producing faster, easier operation schedules with high scheduling flexibility in Industry 4.0 (Liaqait et al., 2021; Parente et al., 2020). Multi-agent systems (MAS) are probably the most common decentralized approaches for scheduling problems.

To the best of the authors' knowledge, no work has been reported towards the development of a decentralized approach for FJSP with setup and transportation times. To fill this gap, a MAS-based approach is proposed for the same problem. The proposed approach consists three classes of agents who cooperatively solve the problem. A Hybrid Grey wolf optimization algorithm (GWO) employed with a critical path-based neighbourhood search is applied for optimization. The performance of the proposed approach is compared with the state-of-the-art centralized approaches by solving 20 standard test instances available in the literature. Our approach has been found to be superior, both in terms of solution quality and convergence. The followings are the main contributions of the current work:

- First, this is the first attempt towards the development of a decentralized approach for solving FJSP with setup time and transportation time.
- Second, a novel approach with only one vector (operation sequence) in place of commonly used two vectors (operation sequence and machine selection) is applied. This approach simplifies the optimization process, especially for large complex problems.
- Third, a novel auctioning mechanism to deal with setup time and transportation time is developed.

The remainder of this paper is organized as follows. Section 2 presents a review of related literature. Section 3 formally defines the problem. Section 4 elaborates on the proposed Multi-agent based approach. Section 5 reports the computational results. Section 6 concludes this research and discusses future research directions.

2. Literature review

The research on FJSP is probably pioneered by Brucker and Schlie (1990) in which the authors used a graphical method to minimize the makespan considering two jobs with the constraints of job precedence and non-overlapping of operations. After that, various metaheuristic approaches, e.g. Tabu search (TS) (Brandimarte, 1993), Genetic Algorithm (GA) (Gao et al., 2006; Kacem et al., 2002a, 2002b; Li & Gao, 2016), Particle Swarm Optimization (PSO) (Ding & Gu, 2020), Finite beam search (FBS) (Wang & Yu, 2010), Grey Wolf Optimization (GWO) (Luo et al., 2019; Zhu & Zhou, 2020), Jaya algorithm (Fan et al., 2021; Li, Deng, et al., 2020), Artificial Bee Colony optimization (ABC) (Li, Huang, et al., 2020; Li et al., 2014), which have been reviewed from time to time e.g. by Demir and İşleyen (2013), Li and Gao (2020), and Zhang, Ding, et al. (2019).

Most of the above articles have considered a centralized decision-making approach in which a single entity takes scheduling decisions. These centralized approaches, however, may lead to higher computing complexity and sub-optimal solutions, especially for large-size FJSPs (Barbati et al., 2012). Apart from centralized approaches, decentralized approaches have also been proposed for FJSP. In the decentralized approach, the problem is decomposed into smaller sub-problems which are solved through a group of entities (also called agents), leading to reduced computational load and higher system flexibility. A multi-agent system is one of the decentralized approaches and is quite popular among researchers. Table 1 shows the literature on multi-agent systems to solve FJSP.

Wu and Weng (2005) proposed a multi-agent scheduling method with two types of agents, i.e. job and machine agents. Job agent, one for each job, is responsible for machine selection based on bids submitted by machine agents. Ennigrou and Ghédira (2008) introduced a multi-agent model consisting of three types of agents: interface agent, job

agents, and resource agents. A job agent selects resources (machines) for its operations as per the availability of the machines. The resource agent checks the feasibility of machine allocations made by all the job agents, and conflicts, if any, are resolved in consultation with the respective job agents. The interface agent and resource agents then use an iterative mechanism for schedule improvement where the interface agent focuses on exploration and the resource agent on exploitation (using TS). Henchiri and Ennigrou (2013) simplified the MAS by removing job agents and using PSO (for global search) and TS (for local search). Nouri et al. (2018, 2016) presented a Holonic multi-agent model where a scheduler agent performs global optimization using GA, and a cluster agent performs a local search using TS. Xiong and Fu (2018) developed a new immune multi-agent scheduling system based on the behaviour of the humoral immunity mechanism. Three agents: shop agent, machine agents and job agents, simulates the functions of peripheral lymphoid tissue, macro-phage and T cells, respectively. Here, the Job agent presents the maximum B cell activation value (based on minimum processing time and completion time) to the corresponding machine agent. Then, MA selects the operation with the maximum B cell activation value. Zhou et al. (2019) proposed a multi-agent based hyper-heuristics integrated with the prior knowledge of the shop floor to solve the online scheduling problem of flexible job shops. Li et al. (2022) proposed a simple two-agent system to solve multi-objective FJSP. These two agents: machine agent and supervision agent (SA) are used to minimize three objectives of makespan, critical machine workload, and total workload. Cooperative and competitive strategies are used to get local solution while adaptive iterative loop solving mechanism is used to get global optimal solution.

In recent years, efforts have been put into making the FJSP more realistic by considering setup time and transportation time. A machine setup may be required at the beginning of a new job, or a job moves from one machine to another. The movement of a job from one machine to another involves transportation time. Recent studies conclude that both the setup and transportation times affect the efficiency of production schedules and give a more realistic view of actual industrial scheduling (Nouri et al., 2016; Zhang et al., 2020). Several approaches have been proposed to solve the FJSP with setup time, FJSP with transportation time and FJSP with both time factors. Table 2 shows the literature for FJSP with setup time, transportation time or both.

As shown in Table 2, five articles consider FJSP with both setup and transportation times. Zhang et al. (2020) has considered three objectives of makespan, total setup time and total transportation time and developed an improved Genetic Algorithm (GA) employed with greedy algorithm. Li, Deng, et al. (2020) consider makespan and total energy consumption objectives and develop an improved Jaya algorithm combined with simulated annealing. Sun et al. (2021) proposed a Hybrid Many-objective Evolutionary Algorithm (HMEA). Zhang et al. (2022) proposed a novel heuristic method using three strategies: population initialization strategy, greedy iterative decoding strategy and local intensification strategy for minimizing makespan and total energy consumption to solve FJSP with Sequence-Dependent Setup Time (SDST) and Transportation times. Li and Lei (2021) proposed an Imperialist Competitive Algorithm (ICA) with feedback for energy efficient FJSP with sequence-dependent setup and transportation times for minimizing makespan, tardiness and energy consumption. It is noted here that a variety of centralized approaches have been proposed for the FJSP with set up and transportation time. The centralized approaches, however, are not suitable for such large complex problems (Zhang, Ding, et al., 2019). To the best of authors knowledge, no attempts have been directed towards development of decentralized approach for the FJSP with set up and transportation time. This study attempts fill this gap and propose a MAS for solving FJSP with setup and transportation times.

Table 1
Literature on multi-agent systems for FJSP.

Problem	Solution approach	Objectives	Author
FJSP	Multi agent model	Earliness-Tardiness	Wu and Weng (2005)
FJSP	Multi agent model + TS	Makespan	Ennigrou and Ghédira (2008)
FJSP	Multi Agent Model + PSO + TS	Makespan	Henchiri and Ennigrou (2013)
FJSP	Holonic Multi Agent Model + GA + TS	Makespan	Nouri et al. (2018)
FJSP with Transportation	Holonic Multi Agent Model + GA + TS	Makespan	Nouri et al. (2016)
FJSP	New immune based multi agent system	Makespan	Xiong and Fu (2018)
FJSP	Multi-agent Based Hyper-heuristics	Mean weighted tardiness, maximum tardiness and mean wait time	Zhou et al. (2019)
FJSP	Simple two-agent system	Makespan, Critical machine workload, total workload	Li et al. (2022)

Table 2
Literature on FJSP with setup time, transportation time or both.

Problem	Solution approach	Objectives	Author
FJSP with setup time	Hybrid GA	Makespan	Wang and Zhu (2021)
	Evolutionary optimization	Makespan	Azzouz et al. (2020)
	Tabu search	Makespan	Shen et al. (2018)
	Iterative local search	Total tardiness	Mousakhani (2013)
FJSP with transportation time	Finite beam search	Makespan	Wang (2010)
	Improved GA	Makespan	Zhang, Sun, et al. (2019)
	Hybrid ICA	Makespan and Total energy consumption (TEC)	Peng et al. (2022)
	hybrid GA	Makespan, Maximum workload and Total workload	Huang and Yang (2019)
	ICA + Simulated annealing	Makespan	Karimi et al. (2017)
FJSP with setup and transportation time	Artificial Immune algorithm	Makespan, TEC	Chen et al. (2020)
	Improved GA	Makespan, Total setup time and Total transportation time	Zhang et al. (2020)
	Improved JAYA	Makespan and Energy consumption	Li, Deng, et al. (2020)
	HMEA	Makespan, Critical workload and Total workload, Penalties of earliness/tardiness	Sun et al. (2021)
	ICA	makespan, Total tardiness and TEC	Li and Lei (2021)
	New Heuristic method	Makespan and Total energy consumption	Zhang et al. (2022)

3. Problem definition

The FJSP considers n jobs that must be processed on m machines. Job i consists of n_i operations with a predefined sequence. Operation O_{ij} is j th operation of job i , which can be assigned to only one of the available machines, k . A machine can perform only one operation at a time. Preemption is not allowed. ST_{ijk} is the setup time needed for setting up machine k for j th operation of job i . Setup time is incurred when job i 's first operation needs to be performed on the machine or two successive operations of job i are processed on different machines. $TT_{g,k}$ is the transportation time to move job i from machine g to machine k for the next operation. The indices and notations for mathematical formulation are as follows:

A. Indices

- i, e : index of jobs, $i = 1, 2, \dots, n$
- j, f : index of operations, $j = 1, 2, \dots, n_i$
- k, g : index of machines, $k = 1, 2, \dots, m$

B. Parameters

- n : Number of jobs
- n_i : Total number of operations for job i
- m : Total number of machines
- O_{ij} : j th operation of i th job
- $O_{i(j-1)}$: $(j-1)$ th operation of i th job, a predecessor of operation O_{ij}
- P_{ijk} : Processing time of the operation O_{ij} on the machine k
- C_{ij} : The completion time of job operation O_{ij}
- $C_{i(j-1)}$: The completion time of job operation $O_{i(j-1)}$
- C_{ef} : The completion time of operation O_{ef} , predecessor operation of operation O_{ij} on the same machine k
- CT_i : Completion time of job i
- ST_{ijk} : Setup time of j th operation of i th job on machine k

- $TT_{g,k}$: Transportation time to move of i th job from machine g to machine k

C. Decision variables

$$\theta_{ijk} = \begin{cases} 1 & \text{if machine } k \text{ is selected for } O_{ij} \\ 0 & \text{Otherwise} \end{cases}$$

$$\theta_{ef,ij,k} = \begin{cases} 1 & \text{if } O_{ef} \text{ is processed before } O_{ij} \text{ on the same machine } k \\ 0 & \text{Otherwise} \end{cases}$$

D. Mathematical Model

$$\min F = \max(CT_i), \quad 1 \leq i \leq n \quad (1)$$

subject to

$$\sum_{k=1}^m \alpha_{ijk} = 1, \forall i, j \quad (2)$$

$$C_{ij} = (P_{ijk} + ST_{ijk} \cdot \theta_{ijk}), \forall i, k; j = 1 \quad (3)$$

$$C_{ij} - C_{i(j-1)} \geq P_{ijk} \cdot \theta_{ijk} + TT_{g,k} \cdot \theta_{i(j-1)g} \cdot \theta_{ijk}, \quad \forall i, g, k; g \neq k; 2 \leq j \leq n_i \quad (4)$$

$$(C_{ij} - C_{ef} - ((P_{ijk} + ST_{ijk}) \cdot \theta_{ijk})) \cdot \phi_{ef,ij,k} \geq 0, \forall i, k; 2 \leq j \leq n_i \quad (5)$$

Eq. (1) is the objective function for minimization of makespan. Eq. (2) ensures the assignment of an operation to one and only one machine. Eq. (3) represents the machine needs only setup before processing the job's first operation. Eq. (4) represents the job precedence constraints. Eq. (5) ensures that only one operation can be performed at a time on any machine. The followings are the assumptions for this problem:

- All jobs and machines are independent of each other and available at time zero.
- Only one operation of a job can be performed on a machine at a time.
- Each machine can process only one operation at a time.
- Pre-emption is not allowed.
- Each job has its own defined order of operations. After completion of one operation of the job, it moves to the next scheduled machine.
- If two consecutive operations of a job use the same machine, both setup and transportation time are null due to no change of machine and no movement between the machines, respectively.

4. The proposed approach

This section explains the proposed Decentralized Multi-agent system to solve the FJSP with setup and transportation times. The job-centred auction mechanism with production reservation (PR) scheme for allocation of the machine to the operation, proposed by Saad et al. (1997), is modified to the problem. Hybrid GWO, a popular swarm-based metaheuristic, is used for optimization. Details of the approach are discussed below.

4.1. Multi-agent architecture

A multi-agent system (MAS) consists of a number of intelligent agents who perceive their environment to exhibit their defined goal or objectives by interacting with other agents (Wooldridge, 2009). As shown in Fig. 1, the proposed architecture of MAS consists of three types of agent classes: Scheduling agent (SA), Job agent (JA) and Machine agent (MA).

4.2. The algorithm

In most of the existing approaches for the FJSP, two vectors: machine selection (MS) and operation sequencing (OS), have been used (Caldeira & Gnanavelbabu, 2019; Li, Deng, et al., 2020; Sun et al., 2021; Zhang et al., 2020, 2021). Contrary to this, only one vector, i.e. operation sequencing (OS), is used in the proposed approach. The operation sequencing (OS) vector is an ordered integer-valued string with length $L = \sum_{i=1}^n n_i$. Fig. 2 illustrates an OS vector for three jobs with three, three and two operations, respectively and also explains the encoding scheme of the OS vector.

Fig. 3 shows the flow chart of the proposed algorithm. Various functions to be performed by the agents and the algorithms are described in the following.

4.2.1. Scheduling agent

The scheduling agent is the main coordinating agent in the proposed MAS. The SA creates the JAs and MAs for the jobs and the machines. The SA also generates Operation Sequence (OS) vectors (explained later) as initial solutions, as shown in Algorithm 1.

Algorithm 1: Main algorithm (Scheduling agent)

```

1 Input:  $n$  (Jobs),  $m$  (Machines), population size ( $N_{pop}$ ), number of iterations ( $N_{iter}$ );
2 Output: Final best OS solution with makespan, total setup and transportation time;
3 Begin:
4 Creation of Job agents and machine agents as per jobs( $n$ ) and machines( $m$ );
5 Population initialization;
6 Randomly generated solutions ( $N_{pop}$ ) sent to JAs for auctioning;
7 for 1 to  $N_{pop}$  do
8   Auctioning process between JAs and MAs (Algorithm 3 & Algorithm 4)
9 Enlist of OS vectors (solutions) with Makespan (fitness) along with total setup and transportation times;
10 Optimization using hybrid Grey Wolf Optimization (hGWO);
11 for 1 to  $N_{iter}$  do
12   Population selection using tournament selection method;
13   Modified discrete search crossover;
14   Adaptive mutation;
15   Selection of best solution(s) based on makespan;
16   Critical path-based neighbourhood search;
17 End

```

4.2.1.1. Population Initialization

In this work, the initial population of OS vectors is generated randomly. These OS vectors are sent to the class of JAs, which generate solutions to the FJSP. The auctioning scheme performed by JAs and MAs for generating solutions is explained in Sections 4.2.2 and 4.2.3.

4.2.1.2. Optimization using hybrid Grey Wolf Optimization

In recent years, Grey Wolf Optimization (GWO) has attracted a widespread interest due to its simplicity, use of few parameters, balanced local and global searching ability, and balanced convergence speed (Jieran et al., 2020; Zhang et al., 2021; Zhu & Zhou, 2020). GWO is found to be competitive to other well-known meta-heuristic algorithms such as differential evolutionary algorithm and particle swarm optimization (Mirjalili et al., 2014) and has been effectively applied to various scheduling problems (Jiang et al., 2018; Qin et al., 2019; Yin et al., 2020) and also FJSP (Jieran et al., 2020; Zhang et al., 2021; Zhu & Zhou, 2020). Inspired by this, we use GWO for the optimization in this work. To further improve its performance, we hybridize it with a critical path neighbourhood search termed hGWO.

The GWO begins with a predetermined pack (population of OS vectors) of wolves of four distinct categories: Alpha wolf (α), Beta (β), Delta (δ), and Omega (ω) wolves following a social hierarchy in the same order. The hGWO follows modified discrete search crossover and adaptive mutation and the critical-path based neighbourhood search as explained next.

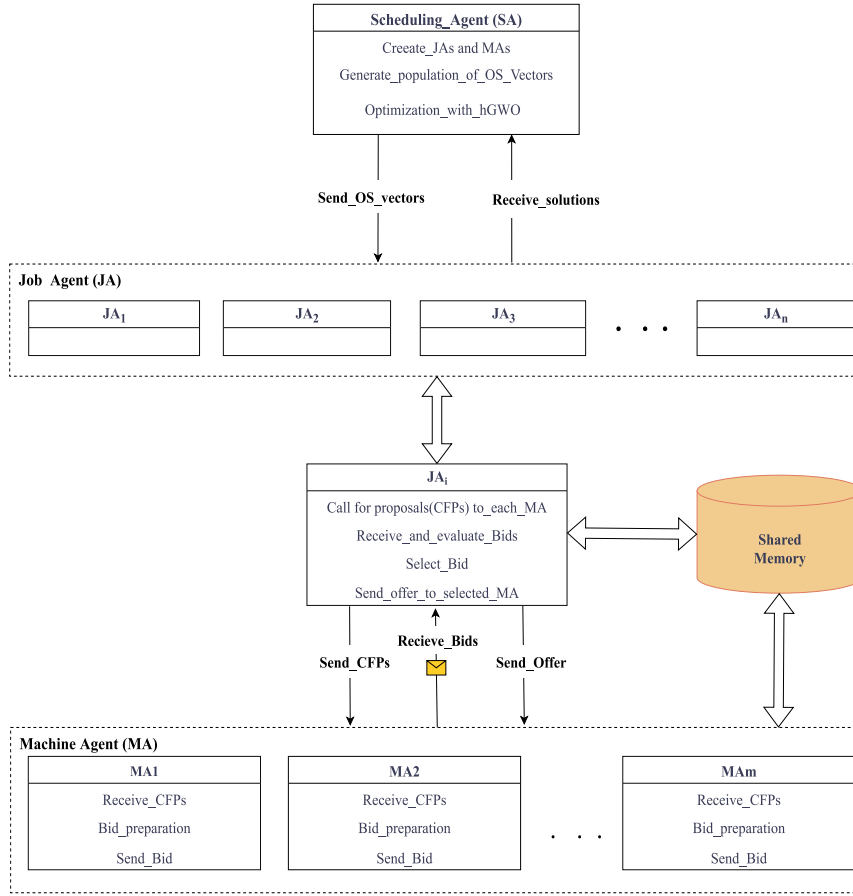


Fig. 1. Block diagram of Multi-agent architecture.

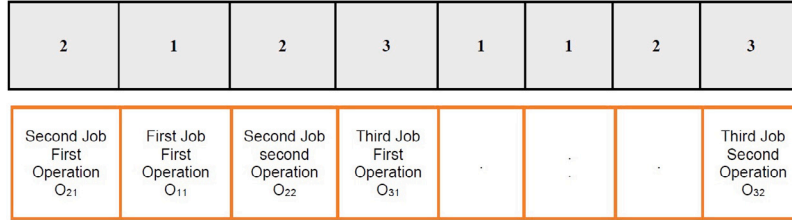


Fig. 2. Encoding scheme of OS vector solution.

a. Selection Operator

Makespan is considered the fitness value of the OS vector represented in Eq. (1). The tournament selection method is used, which is simple and has better or equivalent convergence than other selection operators such as roulette wheel selection, rank based selection and seed selection (Zhang et al., 2020).

b. Modified discrete search crossover

This paper considers a modified discrete search operator developed by Jiang and Zhang (2018) to generate a new set of vectors inheriting the best positional information. A novel uniform precedence order-based crossover operator (fn) with probability (P_{cross}) is employed where one of the three best OS vectors α , β and δ is paired with one of ω (OS vectors) using (r , a random number) as per Eq. (6). t represents current iteration number in Eq. (6).

$$X_w(t+1) = \begin{cases} fn(X_w(t).X_\alpha(t)), & 0 \leq r \leq \frac{1}{3} \\ fn(X_w(t).X_\beta(t)), & \frac{1}{3} \leq r \leq \frac{2}{3} \\ fn(X_w(t).X_\delta(t)), & \frac{2}{3} \leq r \leq 1 \end{cases} \quad (6)$$

Fig. 4 illustrates the mechanism for crossover in which job operations at the odd position of OS vector 1/OS vector 2 are directly transferred to the same position in the new_OS_vector 1/new_OS_vector 2, respectively. The remaining empty positions in the new OS Vectors are filled from OS vector 2/OS vector 1, maintaining the feasibility of the schedule.

c. Adaptive Mutation

Use of best solutions α , β and δ for cross-over may result in loss of population diversity and premature convergence. The adaptive mutation method proposed by Jiang and Zhang (2018) is adopted to avoid this. The mutation rate $d_k(t)$ of k th individual (OS Vector) is determined using Eq. (7) where fitness rank $fit(t) = \frac{1}{fitness}$.

$$d_k(t) = 1 - \frac{Max(fit(t)) - fit(t)}{Max(fit(t)) - Min(fit(t))} \quad (7)$$

Where, ($Max(fit(t))$), ($Min(fit(t))$) and $fit(t)$ indicate the maximum, minimum and individual fitness values of new OS vectors, respectively. As a result, the mutation rate $d_k(t)$ for each solution varies between 0 to 1. For mutation, a random number [0,1] is generated. If the random number is less than the mutation rate, then a mutation operator is chosen

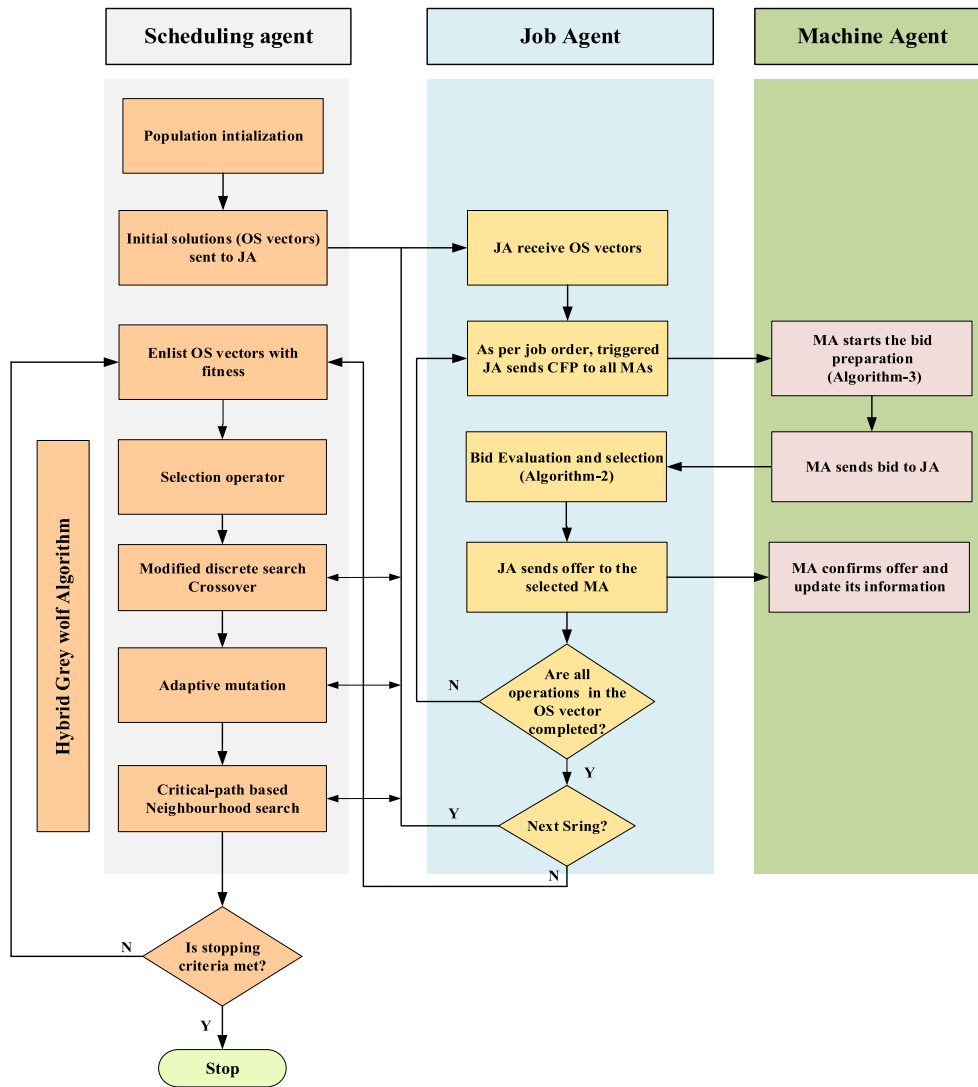


Fig. 3. The flow chart of the proposed algorithm.

from three mutation operators based on the mutation rate($d_k(t)$). Three mutation operators: swap, insertion and reverse and the condition for selection of the respective operator based on mutation rate are given in Figs. 5(a), 5(b) and 5(c) respectively.

d. Critical path-based neighbourhood search

In recent years, various neighbourhood search methods have been hybridized with metaheuristics for the intensive search of the solution space in optimization problems. Critical path-based neighbourhood search has been effectively applied for FJSP (Ennigrou & Ghédira, 2008; Gao et al., 2006; Henchiri & Ennigrou, 2013; Li, Deng, et al., 2020). The critical path is the longest path of job completion times, and the operations belonging to the critical path(s) are called critical operations. In the existing critical path-based neighbourhood search methods, a critical operation is exchanged with a consecutive critical operation of the same machine or reassigned to another available machine (randomly) for the operation. As we are dealing with an OS vector in this work, the critical path-based neighbourhood search is modified in which a critical operation is randomly exchanged with a non-critical operation using swap operator (Fig. 5(a)) as explained in Algorithm 2.

Algorithm 2: Critical path-based neighbourhood search

```

1 Input: Initial solution (best OS Vector(s) of an iteration), critical and non-critical operations of the initial solution, Swap operator, Stopping condition ( $\Delta_{max} = 5$ );
2 Output: New neighbourhood solution ;
3 Begin: Set  $\Delta = 1$ ;
4 while  $\Delta < \Delta_{max}$  do
5   Neighbour solution = Swap(Initial solution);
6   if  $\text{Fitness value}(\text{neighbour solution}) < \text{Fitness value}(\text{Initial solution})$  then
7     Set Initial solution = neighbour solution and set  $\Delta = 1$ 
8   else
9      $\Delta = \Delta + 1$ 
10 End
  
```

4.2.2. Job agent

After receiving an OS vector from the scheduling agent (SA), the JA selects the machine for the operations in the OS vector through auctioning. As per the orders of job operations in the OS vector, the concerned

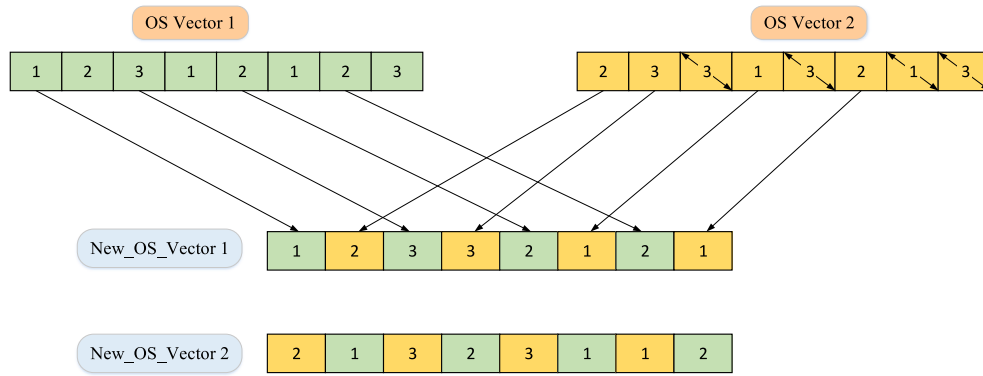


Fig. 4. Uniform POX crossover.

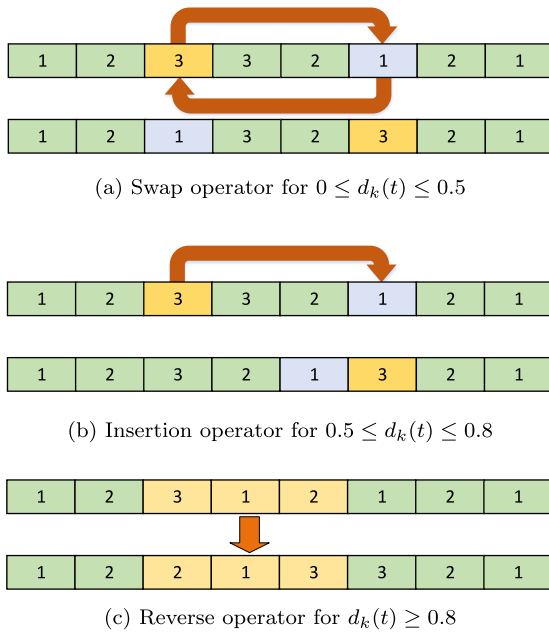


Fig. 5. Convergence Chart of four problem instances.

JA is triggered and starts the auction process by sending a call-for-proposal (CFP) to all MAs, which includes information about the last operation of the job, its completion time and the selected machine for it. In response to CFP, the MAs send bids consisting of expected completion time along with cumulative setup and transportation time till this operation. The JA selects the bids for machine selection in the priority order of completion time, setup time, and transportation time (Algorithm 3).

4.2.3. Machine agent

After receiving CFP from JA, the MAs prepare bids as per Algorithm 4. For preparing the bid, the MA uses information about the machine's last operation number, its completion time along with processing time, setup time and transportation time of the operation for which the bid is to be submitted. It is to be noted that the setup and transportation times are included in bids under certain conditions only. These conditions are illustrated in Fig. 6. The bids sent by the MAs are then evaluated by the

respective JA (Section 4.2.2). Upon receiving the confirmation about the selected bid from JA, the MA updates its information.

Algorithm 3: Job Agent

```

1 Input:  $N_{pop}$  (Population size of solution), Completion time, setup
  time, transport time for the operation;
2 Output: OS vector solution with makespan (fitness), total setup time
  and total transportation time;
3 Begin: /*Step-8 (Algorithm 1)*/;
4 Each OS vector is solved through auction mechanism as follows;
5 for Each operation of an OS Vector do
6   Activated JA sends Call for proposals (CFPs) to all MA(s);
7   Each Machine agent (MA) prepares bid (Algorithm 4);
8   JA received bids with Completion time, setup time and transport
   time;
9   Evaluation of bids with priority criteria ;
10  if Single bid of minimum completion time then
11    Select the bid;
12  else
13    Find bids with minimum setup time of same completion time
    bids;
14    if Single bid of minimum setup time then
15      Select the bid;
16    else
17      Find bids of minimum transport time of same completion
      and setup times;
18      if Single Bid of minimum transportation time then
19        Select the bid;
20      else
21        Select any bid from bids with same value of all three
        time parameters
22  Select bid and send offer to selected MA for confirmation;
23 Similarly, other operations of an OS vector are evaluated in order and
    machines are selected;
24 End

```

5. Computational experience

This section reports a computational experience in which the proposed approach is compared with three state-of-the-art algorithms by solving 20 problem instances.

5.1. Experimental setup

The proposed algorithm is implemented in python programming 3.7.7 with an Intel Core i5-8250U CPU at 3.6 GHz with 8 GB of RAM.

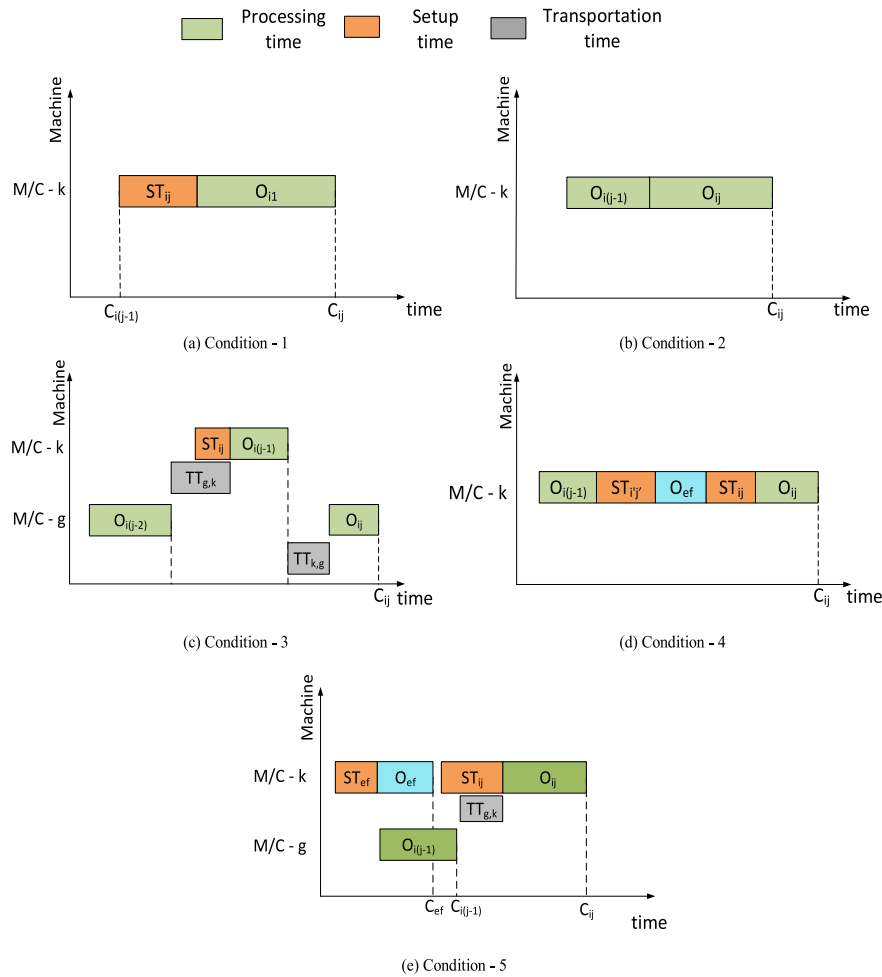


Fig. 6. Gantt charts representing various conditions for inclusion of setup and transportation times (Algorithm 4).

Twenty problem instances have been solved, out of which five instances of size 4×5 , 8×8 , 10×7 , 10×10 , and 15×10 were taken from the literature. These instance were originally developed by Kacem et al. (2002a, 2002b) for FJSP, out of which 4×5 , 8×8 , 10×10 , and 15×10 were modified by Zhang et al. (2020), and one problem 10×7 by the authors to include setup and transportation times. Fifteen instances (numbered MK01–MK15) were generated using problem parameters (number of jobs, number of machines, number of operations in a job, processing times and machine flexibility) as given in Brandimarte (1993). The setup time follows a uniform distribution with minimum as 1 and maximum as the processing time of the respective jobs on the machine. Similarly, transportation times are taken from the uniform distribution between 1 and 5.

As per literature and initial experimentation, the population size, maximum number of iterations, and crossover probability (P_{cross}) are taken as 100, 100, and 0.9, respectively. Each problem instance is solved ten times, and the schedule with the lowest makespan is reported. Total setup time and total transportation time are also reported for the chosen schedule of the lowest makespan. The results obtained by our algorithm and the state-of-the-art algorithms are compared in terms of solution quality and convergence level.

5.2. Results and discussion

In this section, first, the results of our algorithm are compared with the centralized algorithms with consideration of setup time and

transportation times in the FJSP. Second, as none of the existing decentralized approaches has considered setup time and total transportation times, the results of our algorithm are compared with the existing decentralized algorithms for basic FJSP.

5.2.1. Comparison with the centralized algorithms

The results are compared with three centralized algorithms, viz. Enhanced genetic algorithm (EGA) by Dai et al. (2019), Improved Genetic Algorithm (IGA) by Zhang et al. (2020) and Improved JAYA (IJA) algorithm by Li, Deng, et al. (2020). The results are shown in Table 3. In the table, columns 1–3 describe the problems where average flexibility measures the average number of machines that can perform an operation. Columns 4–7 show the makespan obtained by the comparison algorithms and our algorithms. The bold values are the minimum of the makespan obtained by any of the algorithms considered in this study. Columns 8–11 show the deviations (the difference between the makespan obtained by an algorithm and the best makespan obtained by any of the algorithms). Here, the reported deviations are absolute values which may not factor in the problem sizes. Hence a relative measure RPI (Relative Percentage Increase) is also considered (Eq. (8)) as shown in column 12.

$$RPI = \frac{M_c - M_p}{M_c} \times 100 \quad (8)$$

where, M_c is the best makespan by any of the comparison algorithms and M_p is the makespan obtained by our algorithm.

Algorithm 4: Machine Agent

```

1 Input:  $i$  (Auctioneer Job),  $j$  (Operation number of the auctioneer job),  $k$  (Bidder machine),  $P_{ijk}$ (Processing time),  $ST_{ijk}$  (Setup time),  $TT_{g,k}$  (Transportation time),  $LT_{job}$ (Last completion time of auctioneer job), Last_machine (last machine of auctioneer job),  $LT_{machine}$  (Last completion time of bidder machine), Last_job (Last job of bidder machine), Completion_time (Completion time of the operation), Setup_time (Cumulative setup time) and Transportation_time (Cumulative transportation time) for the operation;
2 Output: Bid with Completion_time, Setup_time and Transportation_time ;
3 Begin: Get Call for proposal (CFP) from the JA;
4 MA starts the bid preparation;
5 if Idle time duration is available on the MA, then
6   if the operation considering setup and transportation time factors can be inserted during idle time then
7     if the job before idle time and the auctioneer job are same then
8       Completion_time = max (Start time of idle duration,  $LT_{job} + TT_{g,k}$ ) +  $P_{ijk}$  ;
9       Transportation_time = Transportation_time +  $TT_{g,k}$ ;
10    else
11      Completion_time = max (Start time of idle duration +  $ST_{ijk}$ ,  $LT_{job} + TT_{g,k}$ ) +  $P_{ijk}$  ;
12      Setup_time = Setup_time +  $ST_{ijk}$ ;
13      Transportation_time = Transportation_time +  $TT_{g,k}$ ;
14    else
15      Step:(17-33)
16  else
17    if The operation is first operation of the auctioneer job (Fig. 6a) then
18      Completion_time =  $ST_{ijk} + P_{ijk}$ ;
19      Setup_time = Setup_time +  $ST_{ijk}$ ;
20    else
21      if Last_machine = bidder machine & Last_job = auctioneer job (Fig. 6b) then
22        Completion_time =  $LT_{machine} + P_{ijk}$ ;
23      else if Last_machine  $\neq$  bidder machine & Last_job = auctioneer job (Fig. 6c) then
24        Completion_time =  $LT_{job} + P_{ijk} + TT_{g,k}$ ;
25        Transportation_time = Transportation_time +  $TT_{g,k}$ ;
26      else if Last_machine = bidder machine & Last_job  $\neq$  auctioneer job (Fig. 6d) then
27        Completion_time =  $LT_{machine} + P_{ijk} + ST_{ijk}$ ;
28        Setup_time = Setup_time +  $ST_{ijk}$ ;
29      else if Last_machine  $\neq$  bidder machine & Last_job  $\neq$  auctioneer job (Fig. 6e) then
30        Completion_time = max (  $LT_{machine} + ST_{ijk}$ ,  $LT_{job} + TT_{g,k}$ ) +  $P_{ijk}$ ;
31        Setup_time = Setup_time +  $ST_{ijk}$ ;
32        Transportation_time = Transportation_time +  $TT_{g,k}$ ;
33 End

```

It can be seen from Table 3 that the makespan obtained by our algorithm are better than the state-of-the-art algorithms for the 17 problem instances and competitive for 2 instances. The box-plot (Fig. 7) shows the consistent performance of our algorithm. The last row of deviations columns show the mean of deviations of a algorithm. Our algorithm with 1.50 mean deviation outperforms all the state-of-the-art algorithms. From RPI (column 12), it can be seen that our algorithm is better than the best of the comparison algorithms to the extent of 35% with an average of 10.27%. As it is not possible to show Gantt charts for all the problems we show them for four problems 10 $\times$ 7, 15 $\times$ 10, MK02, and MK04 (Figs. 8(a), 8(b), 9, and 10). The bars in orange show the setup, while the bars in grey show the transportation of respective job operations.

The total setup time and total transportation time for the chosen schedule for the lowest makespan of a algorithm are reported in Table 4. It is to be noted that the values in bold are the minimum values of the total setup time and total transportation time obtained by any of the algorithms considered in this study. It can be seen from Table 4 that other than makespan, our algorithm also proves effectiveness by achieving 11(times) and 7(times) best results (in bold), respectively for total setup and transportation times than the comparison algorithms.

Convergence is also an important criterion for the comparison of algorithms. Thus, the convergence curves are also shown for the problems

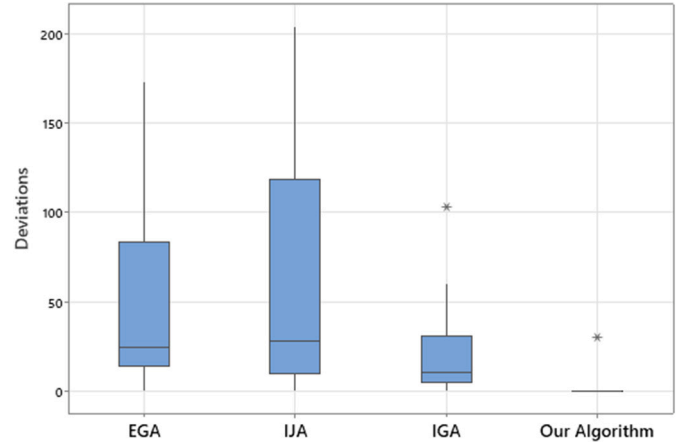


Fig. 7. Box-plot of the distribution of deviations of the algorithms.

8 $\times$ 8, 15 $\times$ 10, MK02, MK06, MK10 and MK15 (Fig. 11(a), 11(b), 11(c), 11(d), 11(e) and 11(f)). It can be seen that our algorithm converges faster than all of the comparison algorithms. Thus, our algorithm not only provides better quality of solution but is also faster.

5.2.2. Comparison with the existing decentralized algorithms

In this section the proposed approach is compared with existing decentralized algorithms. It is, however, noted that none of the decentralized approaches addresses the issues of set up and transportation time in scheduling of FJSP. Thus, to make a fair comparison, our algorithm has been run for basic FJSP, i.e., without set up and transportation times. Four existing decentralized algorithms: MAT-SLO+(Ennigrou & Ghédira, 2008), MATSPSO (Henchiri & Ennigrou, 2013), NIMASS (Xiong & Fu, 2018) and GATS+HM (Nouri et al., 2018). developed for basic FJSP, have been considered in this comparison. Table 5 shows the makespan obtained by the four comparison algorithms and our algorithm for the 15 problem instances considered in the research. As done in the previous section, deviation (the difference of best makespan and makespan obtained by the respective algorithm) is also determined. It can be seen that, the average deviation of 1.9 obtained from our algorithm is far better than the three algorithms and comparable with the 0.46 obtained by NIMASS.

6. Conclusion and future work

Flexible job shops are typical in manufacturing in which a machine is flexible to perform various operations on jobs. However, the scheduling problem of flexible job shops (called FJSP) is more complex than traditional job shops. The complexity of the problem is further increased due to consideration of setup and transportation times while scheduling. Most of the existing approaches for solving FJSP with setup and transportation times assume centralized decision-making where all scheduling decisions are taken by a single central decision entity. These centralized algorithms suffer from high computational effort and slower convergence, especially for large and complex problems.

The decentralized decision-making has become a de-facto approach for solving complex scheduling problems. Despite this, no effort has been put into developing a decentralized approach for FJSP with setup and transportation time. A Multi-Agent System (MAS), one of the most popular decentralized decision-making approaches, is developed in this work. The proposed multi-agent algorithm considers three classes of cooperative agents: scheduling agent, job agent and machine agent. A single OS vector representing a solution is used in contrast to two vectors: OS (operations sequence) and MS (machine selection) of the

Table 3
Comparison of the results for the algorithms.

Problem instance	n × m	Average flexibility	Makespan				Deviation				RPI
			EGA	IJA	IGA	Our algorithm	EGA	IJA	IGA	Our algorithm	
(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
4 × 5	4 × 5	Full	23	16	18	16	7	0	2	0	0
8 × 8	8 × 8	6.41	37	32	29	23	14	9	6	0	21
10 × 7	10 × 7	Full	34	25	20	17	17	8	3	0	15
10 × 10	10 × 10	Full	25	23	17	11	14	12	6	0	35
15 × 10	15 × 10	Full	48	34	28	20	28	14	8	0	29
MK01	10 × 6	2	74	88	71	66	8	22	5	0	7
MK02	10 × 6	3.5	63	52	50	45	18	7	5	0	10
MK03	15 × 8	3	226	226	226	226	0	0	0	0	0
MK04	15 × 8	2	119	134	118	104	15	30	14	0	12
MK05	15 × 4	1.5	297	302	293	276	21	26	17	0	6
MK06	10 × 15	3	163	191	123	109	54	82	33	0	23
MK07	20 × 5	3	257	244	234	214	43	30	20	0	9
MK08	20 × 10	1.5	803	873	803	794	9	79	9	0	1
MK09	20 × 10	3	534	611	520	460	74	151	60	0	12
MK10	20 × 15	3	443	533	400	348	95	185	52	0	13
MK11	30 × 5	1.5	1118	1131	1000	1030	118	131	0	30	−3
MK12	30 × 10	2	770	742	739	727	43	15	12	0	2
MK13	30 × 10	3	641	631	578	554	87	77	24	0	4
MK14	30 × 15	3	742	786	708	605	137	181	103	0	15
MK15	30 × 15	5	694	725	561	515	179	210	46	0	8
Mean			355.7	351.26	326.80	308	49.05	63.45	20.30	1.50	10.27

Table 4
Comparison of the results for setup and transportation times.

Problem instance	Total setup time				Total transportation time			
	EGA	IJA	IGA	Our algorithm	EGA	IJA	IGA	Our algorithm
4 × 5	13	9	8	9	4	2	4	2
8 × 8	51	40	33	40	44	31	33	27
10 × 7	59	30	27	18	45	16	26	16
10 × 10	42	26	27	23	44	10	8	8
15 × 10	116	65	54	34	99	54	44	35
MK01	104	115	98	97	93	92	69	98
MK02	87	67	69	65	81	57	64	64
MK03	368	412	324	257	218	212	228	190
MK04	199	205	175	196	171	174	140	166
MK05	343	334	339	327	205	182	175	209
MK06	276	328	262	250	285	369	331	283
MK07	335	287	266	272	161	122	169	164
MK08	1337	1376	1195	1333	499	534	520	550
MK09	1134	1428	936	1191	534	553	440	501
MK10	994	1146	821	847	591	621	565	602
MK11	1678	1826	1539	1650	394	452	417	450
MK12	2073	2138	1990	2016	430	501	454	484
MK13	1582	1535	1491	1490	413	398	467	448
MK14	2435	2314	2495	2325	806	743	712	796
MK15	2177	2774	2103	1830	796	812	823	782
Mean	770.15	822.75	711.50	713.50	295.65	296.75	284.45	293.75

existing approaches. Machine selection in our algorithm is carried out by a novel auction mechanism which takes care of setup and transportation times while scheduling. The hybrid Grey wolf algorithm (hGWO) with novel critical path-based neighbourhood search is employed for optimization.

To test the performance of the proposed algorithm, results are compared with three state-of-the-art algorithms: EGA, IGA and IJA. Our algorithm has outperformed the comparison algorithms on most problem instances in minimizing the makespan. The proposed algorithm provides schedules with effective total setup and total transportation times. Our algorithm's convergence has also been faster than the comparison algorithms. Performance of our algorithm has also been found to be better than most of the decentralized algorithms. Our approach can be useful for developing intelligent decentralized scheduling systems, which are crucial for Industry 4.0.

In future research, a multi-agent approach for multi-objective FJSP with setup and transportation time with factors like varying processing

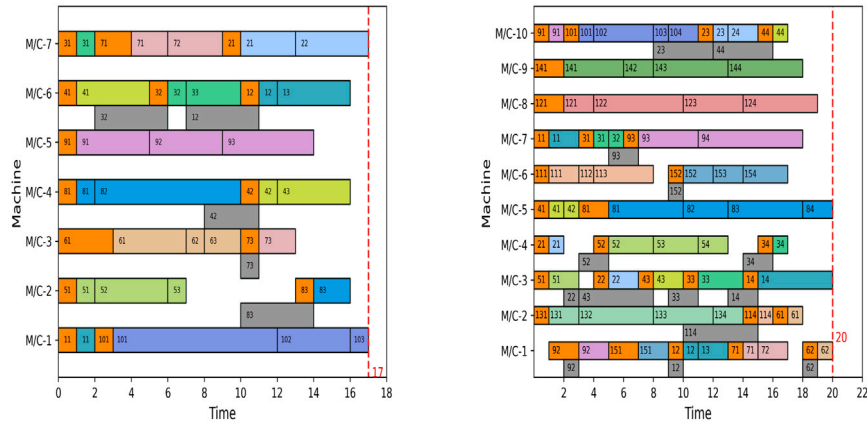
speeds, energy consumption and/or machine availability can be considered. The application of Machine Learning (ML) and Deep learning (DL) algorithms can also be explored.

CRediT authorship contribution statement

Manojkumar Pal: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. **Murari Lal Mittal:** Writing – review & editing, Validation, Supervision. **Gunjan Soni:** Supervision. **Satyendra S. Chouhan:** Editing, Supervision. **Manish Kumar:** Writing–review, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.



(a) Gantt chart of Instance (10×7) with Makespan = 17, total setup time = 18 and total transportation time = 16

(b) Gantt chart of Instance (15×10) with Makespan = 20, total setup time = 34 and total transportation time = 35

Fig. 8. The Gantt charts for two problem instances (10×7) and (15×10) .

Table 5

Comparison of the makespan results for decentralized algorithms.

Problem instance	Best makespan	MATSLO+		MATSPSO		NIMASS		GATS+HM		Our algorithm	
		Makespan	Deviation	Makespan	Deviation	Makespan	Deviation	Makespan	Deviation	Makespan	Deviation
4×5	11	—	—	—	—	—	—	11	0	11	0
8×8	14	—	—	—	—	14	0	14	0	14	0
10×7	11	—	—	—	—	—	—	11	0	11	0
10×10	7	—	—	—	—	7	0	7	0	7	0
15×10	11	—	—	—	—	11	0	11	0	11	0
MK01	39	40	1	39	0	40	1	42	3	40	1
MK02	27	32	5	27	0	28	1	32	5	28	1
MK03	204	—	—	207	3	204	0	211	7	204	0
MK04	65	67	2	65	0	65	0	81	16	66	1
MK05	174	188	14	174	0	177	3	186	12	179	5
MK06	66	85	19	72	6	67	1	86	20	66	0
MK07	144	154	10	154	10	144	0	157	13	149	5
MK08	523	523	0	523	0	523	0	523	0	523	0
MK09	312	437	125	340	28	312	0	369	57	312	0
MK10	229	380	151	299	70	229	0	296	67	235	6
Average Deviation			32.7		11.7		0.46		13.33		1.9

Note: “—”: Not given by author.

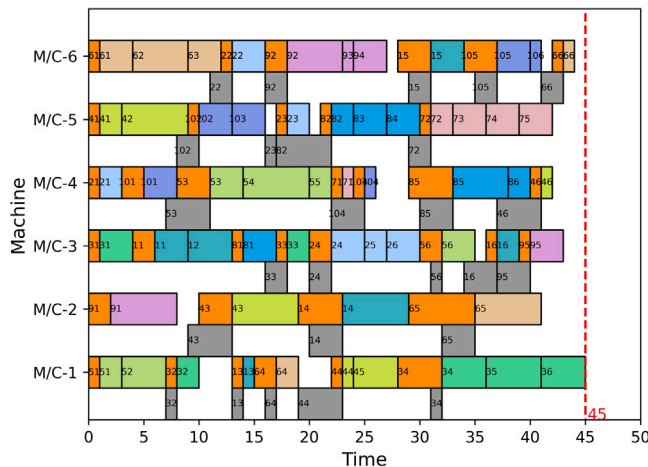


Fig. 9. The Gantt chart for the problem instance MK02.

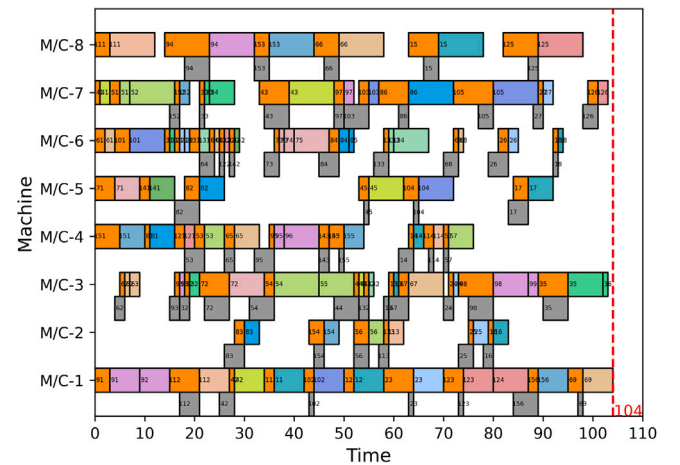


Fig. 10. The Gantt chart for the problem instance MK04.

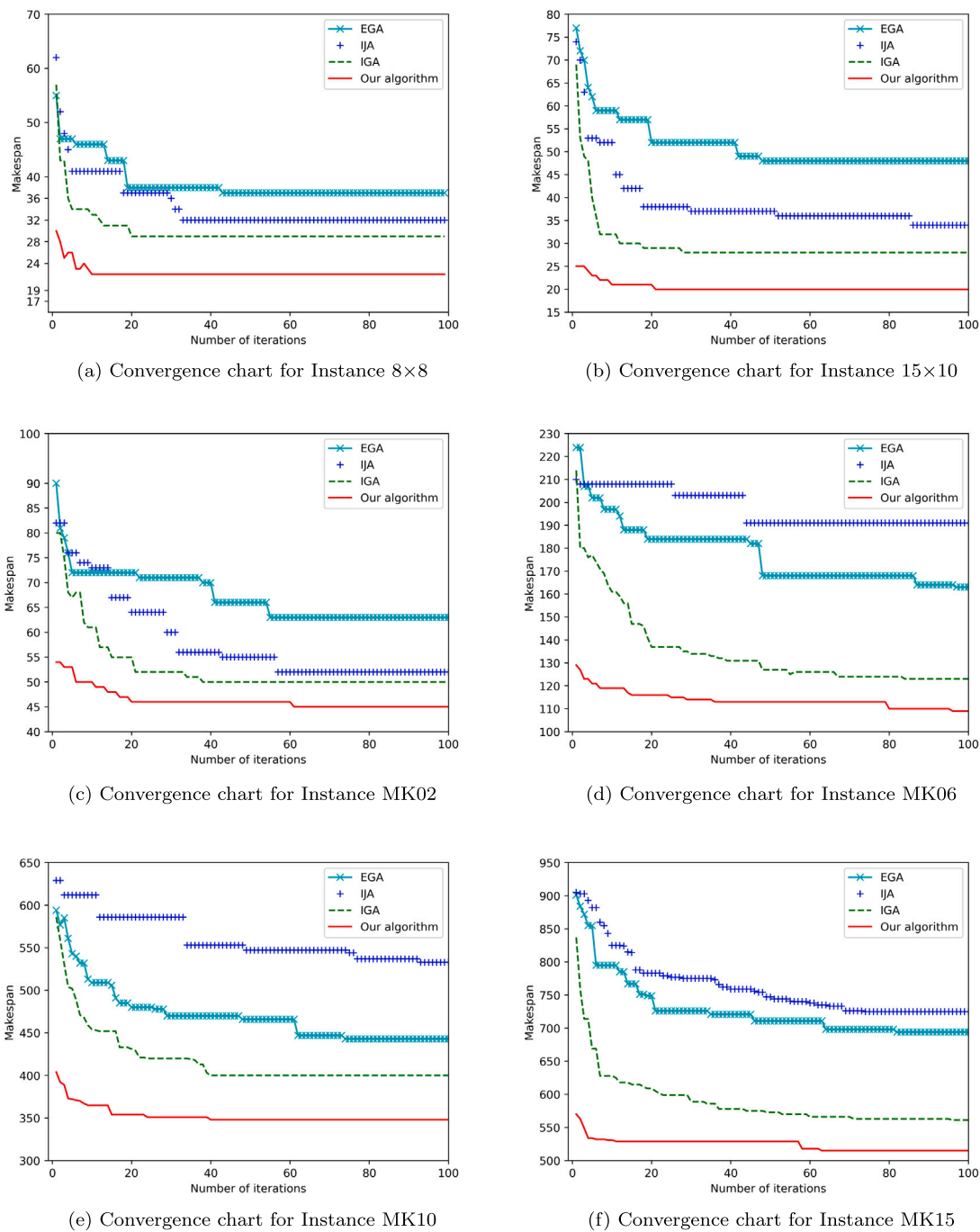


Fig. 11. Convergence Charts of six problem instances.

Data availability

Data will be made available on request.

References

- Azzouz, A., Chaabani, A., Ennigrou, M., & Said, L. B. (2020). Handling sequence-dependent setup time flexible job shop problem with learning and deterioration considerations using evolutionary bi-level optimization. *Applied Artificial Intelligence*, 34(6), 433–455.
- Barbati, M., Bruno, G., & Genovese, A. (2012). Applications of agent-based models for optimization problems: A literature review. *Expert Systems with Applications*, 39(5), 6020–6028.
- Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183.
- Brucker, P., & Schlie, R. (1990). Job-shop scheduling with multi-purpose machines. *Computing*, 45(4), 369–375.
- Caldeira, R. H., & Gnanavelbabu, A. (2019). Solving the flexible job shop scheduling problem using an improved Jaya algorithm. *Computers & Industrial Engineering*, 137, Article 106064.
- Chen, X.-L., Li, J.-q., Han, Y.-y., & Sang, H.-y. (2020). Improved artificial immune algorithm for the flexible job shop problem with transportation time. *Measurement and Control*, 53(9–10), 2111–2128.
- Dai, M., Tang, D., Giret, A., & Salido, M. A. (2019). Multi-objective optimization for energy-efficient flexible job shop scheduling problem with transportation constraints. *Robotics and Computer-Integrated Manufacturing*, 59, 143–157.
- Demir, Y., & İşleyen, S. K. (2013). Evaluation of mathematical models for flexible job-shop scheduling problems. *Applied Mathematical Modelling*, 37(3), 977–988.
- Ding, H., & Gu, X. (2020). Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem. *Computers & Operations Research*, 121, Article 104951.

- Ennigrou, M., & Ghédira, K. (2008). New local diversification techniques for flexible job shop scheduling problem with a multi-agent approach. *Autonomous Agents and Multi-Agent Systems*, 17(2), 270–287.
- Fan, J., Shen, W., Gao, L., Zhang, C., & Zhang, Z. (2021). A hybrid Jaya algorithm for solving flexible job shop scheduling problem considering multiple critical paths. *Journal of Manufacturing Systems*, 60, 298–311.
- Fattahi, P., Jolai, F., & Arkat, J. (2009). Flexible job shop scheduling with overlapping in operations. *Applied Mathematical Modelling*, 33(7), 3076–3087.
- Gao, J., Gen, M., & Sun, L. (2006). Scheduling jobs and maintenances in flexible job shop with a hybrid genetic algorithm. *Journal of Intelligent Manufacturing*, 17(4), 493–507.
- Henchiri, A., & Ennigrou, M. (2013). Particle swarm optimization combined with tabu search in a multi-agent model for flexible job shop problem. In *International conference in swarm intelligence* (pp. 385–394). Springer.
- Huang, X., & Yang, L. (2019). A hybrid genetic algorithm for multi-objective flexible job shop scheduling problem considering transportation time. *International Journal of Intelligent Computing and Cybernetics*, 12(2), 154–174.
- Jiang, T., & Zhang, C. (2018). Application of grey wolf optimization for solving combinatorial problems: Job shop and flexible job shop scheduling cases. *Ieee Access*, 6, 26231–26240.
- Jiang, T., Zhang, C., Zhu, H., & Deng, G. (2018). Energy-efficient scheduling for a job shop using grey wolf optimization algorithm with double-searching mode. *Mathematical Problems in Engineering*, 2018.
- Jieran, Y., Aimin, W., Yan, G., & Xinyi, S. (2020). An improved grey wolf optimizer for flexible job-shop scheduling problem. In *2020 IEEE 11th international conference on mechanical and intelligent manufacturing technologies* (pp. 213–217). IEEE.
- Kacem, I., Hammadi, S., & Borne, P. (2002a). Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 32(1), 1–13.
- Kacem, I., Hammadi, S., & Borne, P. (2002b). Pareto-optimality approach for flexible job-shop scheduling problems: hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation*, 60(3–5), 245–276.
- Karimi, S., Ardalan, Z., Naderi, B., & Mohammadi, M. (2017). Scheduling flexible job-shops with transportation times: Mathematical models and a hybrid imperialist competitive algorithm. *Applied Mathematical Modelling*, 41, 667–682.
- Li, Deng, J.-w., Li, C.-y., Han, Y.-y., Tian, J., Zhang, B., & Wang, C.-g. (2020). An improved jaya algorithm for solving the flexible job shop scheduling problem with transportation and setup times. *Knowledge-Based Systems*, 200, Article 106032.
- Li, X., & Gao, L. (2016). An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *International Journal of Production Economics*, 174, 93–110.
- Li, X., & Gao, L. (2020). Review for flexible job shop scheduling. *Effective Methods for Integrated Process Planning and Scheduling*, 17–45.
- Li, Y., Huang, W., Wu, R., & Guo, K. (2020). An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem. *Applied Soft Computing*, 95, Article 106544.
- Li, M., & Lei, D. (2021). An imperialist competitive algorithm with feedback for energy-efficient flexible job shop scheduling with transportation and sequence-dependent setup times. *Engineering Applications of Artificial Intelligence*, 103, Article 104307.
- Li, J.-Q., Pan, Q.-K., & Tasgetiren, M. F. (2014). A discrete artificial bee colony algorithm for the multi-objective flexible job-shop scheduling problem with maintenance activities. *Applied Mathematical Modelling*, 38(3), 1111–1132.
- Li, Y., Wang, J., & Liu, Z. (2022). A simple two-agent system for multi-objective flexible job-shop scheduling. *Journal of Combinatorial Optimization*, 43(1), 42–64.
- Liaqait, R. A., Hamid, S., Warsi, S. S., & Khalid, A. (2021). A critical analysis of job shop scheduling in context of industry 4.0. *Sustainability*, 13(14), 7684.
- Luo, S., Zhang, L., & Fan, Y. (2019). Energy-efficient scheduling for multi-objective flexible job shops with variable processing speeds by grey wolf optimization. *Journal of Cleaner Production*, 234, 1365–1384.
- Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, 69, 46–61.
- Mousakhani, M. (2013). Sequence-dependent setup time flexible job shop scheduling problem to minimise total tardiness. *International Journal of Production Research*, 51(12), 3476–3487.
- Nouri, H. E., Belkahlia Driss, O., & Ghédira, K. (2018). Solving the flexible job shop problem by hybrid metaheuristics-based multiagent model. *Journal of Industrial Engineering International*, 14(1), 1–14.
- Nouri, H. E., Driss, O. B., & Ghédira, K. (2016). Simultaneous scheduling of machines and transport robots in flexible job shop environment using hybrid metaheuristics based on clustered holonic multiagent model. *Computers & Industrial Engineering*, 102, 488–501.
- Parente, M., Figueira, G., Amorim, P., & Marques, A. (2020). Production scheduling in the context of industry 4.0: Review and trends. *International Journal of Production Research*, 58(17), 5401–5431.
- Peng, Z., Zhang, H., Tang, H., Feng, Y., & Yin, W. (2022). Research on flexible job-shop scheduling problem in green sustainable manufacturing based on learning effect. *Journal of Intelligent Manufacturing*, 33(6), 1725–1746.
- Qin, H., Fan, P., Tang, H., Huang, P., Fang, B., & Pan, S. (2019). An effective hybrid discrete grey wolf optimizer for the casting production scheduling problem with multi-objective and multi-constraint. *Computers & Industrial Engineering*, 128, 458–476.
- Saad, A., Kawamura, K., & Biswas, G. (1997). Performance evaluation of contract net-based hierarchical scheduling for flexible manufacturing systems. *Intelligent Automation & Soft Computing*, 3(3), 229–247.
- Shen, L., Dauzère-Pérès, S., & Neufeld, J. S. (2018). Solving the flexible job shop scheduling problem with sequence-dependent setup times. *European Journal of Operational Research*, 265(2), 503–516.
- Sun, J., Zhang, G., Lu, J., & Zhang, W. (2021). A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times. *Computers & Operations Research*, 132, Article 105263.
- Wang, S. (2010). Filtered beam search based flexible job shop scheduling problem with transportation time. *Advanced Materials Research*, 97–101, 2440–2443.
- Wang, S., & Yu, J. (2010). An effective heuristic for flexible job-shop scheduling problem with maintenance activities. *Computers & Industrial Engineering*, 59(3), 436–447.
- Wang, Y., & Zhu, Q. (2021). A hybrid genetic algorithm for flexible job shop scheduling problem with sequence-dependent setup times and job lag times. *Ieee Access*, 9, 104864–104873.
- Wooldridge, M. (2009). *An introduction to multiagent systems*. John Wiley & sons.
- Wu, Z., & Weng, M. X. (2005). Multiagent scheduling method with earliness and tardiness objectives in flexible job shops. *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, 35(2), 293–301.
- Xiong, W., & Fu, D. (2018). A new immune multi-agent system for the flexible job shop scheduling problem. *Journal of Intelligent Manufacturing*, 29(4), 857–873.
- Yin, L., Zhuang, M., Jia, J., & Wang, H. (2020). Energy saving in flow-shop scheduling management: An improved multiobjective model based on grey wolf optimization algorithm. *Mathematical Problems in Engineering*, 2020.
- Zhang, J., Ding, G., Zou, Y., Qin, S., & Fu, J. (2019). Review of job shop scheduling research and its new perspectives under industry 4.0. *Journal of Intelligent Manufacturing*, 30(4), 1809–1830.
- Zhang, G., Hu, Y., Sun, J., & Zhang, W. (2020). An improved genetic algorithm for the flexible job shop scheduling problem with multiple time constraints. *Swarm and Evolutionary Computation*, 54, Article 100664.
- Zhang, G., Sun, J., Liu, X., Wang, G., & Yang, Y. (2019). Solving flexible job shop scheduling problems with transportation time based on improved genetic algorithm. *Mathematical Biosciences and Engineering*, 16(3), 1334–1347.
- Zhang, C., Wang, K., Ma, Q., Li, X., & Gao, L. (2021). A discrete grey wolf optimizer for solving flexible job shop scheduling problem with lot-streaming. In *2021 IEEE 24th international conference on computer supported cooperative work in design* (pp. 969–974). IEEE.
- Zhang, H., Xu, G., Pan, R., & Ge, H. (2022). A novel heuristic method for the energy-efficient flexible job-shop scheduling problem with sequence-dependent set-up and transportation time. *Engineering Optimization*, 54(10), 1646–1667.
- Zhou, Y., Yang, J.-J., & Zheng, L.-Y. (2019). Multi-agent based hyper-heuristics for multi-objective flexible job shop scheduling: A case study in an aero-engine blade manufacturing plant. *Ieee Access*, 7, 21147–21176.
- Zhu, Z., & Zhou, X. (2020). An efficient evolutionary grey wolf optimizer for multi-objective flexible job shop scheduling problem with hierarchical job precedence constraints. *Computers & Industrial Engineering*, 140, Article 106280.